Labo 14 — Labo pratique : configurer la stack sans reconstruire

Objectif : créer des ConfigMaps depuis un manifeste et depuis des fichiers, un Secret, les injecter en variables et en fichiers montés dans l'API et dans un nginx officiel non modifié, puis observer ce qu'une modification change — ou ne change pas — dans les Pods en cours.

Prérequis — Labo 13 terminé : k3s, `localhost/api:1.0` dans le cluster.

Fichiers fournis (`files/`) - `configmap-api.yaml` — `APP_MESSAGE` et `APP_PROFILE` pour l'API. - `secret-db.yaml` — le mot de passe de la base, en `stringData` . - `deployment-api.yaml` — l'API : `envFrom` la ConfigMap, le Secret monté sous `/run/secrets/db` . - `deployment-web.yaml` — `nginx:alpine` tel quel, configuré par deux ConfigMaps montées, plus les Services `web` et `api` . - `nginx-proxy.conf` , `index.html` — les fichiers qui deviendront des ConfigMaps.

Étape 1 — Des ConfigMaps, de deux façons

```
mkdir -p ~/labo-docker/14 && cd ~/labo-docker/14 && cp <chemin-du-labo>/files/* .
kubectl create namespace lab14 ; kubectl config set-context --current --namespace=lab14
kubectl apply -f configmap-api.yaml
kubectl create configmap web-nginx --from-file=default.conf=nginx-proxy.conf
kubectl create configmap web-html --from-file=index.html
kubectl get configmaps
kubectl get configmap api-config -o yaml | sed -n '/^data:/,/^kind/p'
kubectl describe configmap web-nginx | sed -n '/^Data/,/^$/p' | head -8
```

Observez quatre ConfigMaps (dont `kube-root-ca.crt` , présente dans tout namespace), `api-config` avec ses deux clés, et `web-nginx` dont la clé `default.conf` contient le fichier nginx entier.

Explication. Une ConfigMap est un dictionnaire : une valeur peut être un mot ou un fichier de vingt lignes. `--from-file=cle=fichier` choisit le nom de la clé — ici le nom que le fichier devra porter dans le conteneur.

Étape 2 — Un Secret, et ce que vaut son « chiffrement »

```
kubectl apply -f secret-db.yaml
kubectl get secret db-secret
kubectl get secret db-secret -o yaml | grep -A1 '^data:'
kubectl get secret db-secret -o jsonpath='{.data.POSTGRES_PASSWORD}' | base64 -d ; echo
kubectl create secret generic demo --from-literal=token=abc --dry-run=client -o yaml | grep -A1
'^data:'
```

Observez `db-secret Opaque 1` , la valeur `POSTGRES_PASSWORD: UzNjcjN0LXBn` , puis `S3cr3t-pg` rendu par `base64 -d` ; et un manifeste produit sans rien créer, avec `token: YWJj` .

Explication. `stringData` est converti en base64 par l'API server ; quiconque peut lire le Secret peut le décoder. La protection vient du RBAC, du chiffrement au repos et de ce qu'on ne commite jamais ce YAML en clair.

Étape 3 — Injecter : variables et fichiers

```
kubectl apply -f deployment-api.yaml -f deployment-web.yaml
kubectl rollout status deployment/api --timeout=60s ; kubectl rollout status deployment/web --
timeout=90s
kubectl exec deploy/api -- env | grep -E 'APP_|DB_'
kubectl exec deploy/api -- ls -l /run/secrets/db/
kubectl exec deploy/api -- cat /run/secrets/db/POSTGRES_PASSWORD ; echo
kubectl exec deploy/api -- wget -qO- http://localhost:8080/ ; echo
```

Observez `APP_MESSAGE=bonjour depuis une ConfigMap`, `APP_PROFILE=staging` (depuis `envFrom`) et `DB_PASSWORD_FILE=/run/secrets/db/POSTGRES_PASSWORD` — mais **pas** le mot de passe dans l'environnement ; un fichier `POSTGRES_PASSWORD -> ..data/POSTGRES_PASSWORD` monté en lecture seule ; `S3cr3t-pg` quand on l'ouvre ; et l'API qui répond avec le message de la ConfigMap.

```
kubectl exec deploy/web -- ls -l /etc/nginx/conf.d/ /usr/share/nginx/html/
kubectl port-forward svc/web 18080:80 > /dev/null 2>&1 &
sleep 2 ; curl -s localhost:18080/api/ ; echo ; kill %1
```

Observez dans le Pod `web` — l'image `nginx:alpine` **officielle** — un `default.conf -> ..data/default.conf` et un `index.html -> ..data/index.html` venus des ConfigMaps, et la chaîne complète qui répond via le proxy.

Explication. Variables pour l'API, fichiers pour nginx et pour le secret. Plus d'image `web:1.0` ni de `hostPath` : la configuration est un objet du cluster, disponible sur tout nœud.

Étape 4 — Modifier une ConfigMap

```
kubectl patch configmap api-config -p '{"data":{"APP_MESSAGE":"message modifie"}}'
sleep 5 ; kubectl exec deploy/api -- env | grep APP_MESSAGE
kubectl rollout restart deployment/api ; kubectl rollout status deployment/api --timeout=60s
kubectl exec deploy/api -- env | grep APP_MESSAGE
```

Observez après le `patch`, toujours `bonjour depuis une ConfigMap` dans les Pods en cours ; après `rollout restart`, `message modifie`.

```
kubectl patch configmap web-html -p '{"data":{"index.html":"<h1>index modifie par la
ConfigMap</h1>\n"}}'
sleep 75 ; kubectl exec deploy/web -- cat /usr/share/nginx/html/index.html
```

Observez, sans aucun redémarrage, `<h1>index modifie par la ConfigMap</h1>` (comptez une à deux minutes selon la synchronisation du kubelet).

Explication. Une variable est copiée dans le processus à sa création ; un volume ConfigMap est un lien vers un répertoire que le kubelet remplace. Pour les variables, `rollout restart` ; pour les fichiers, de la patience — et une application qui relit le fichier.

Étape 5 — Une clé qui n'existe pas

```
kubectl patch deployment api --type=json -
p='[{"op":"add","path":"/spec/template/spec/containers/0/env/-","value":
{"name":"APP_PROFILE","valueFrom":{"configMapKeyRef":{"name":"api-config","key":"profil"}}}}]'
sleep 8 ; kubectl get pods | grep api
kubectl describe pods -l app=api | grep "couldn't find key" | tail -1
kubectl rollout undo deployment/api ; kubectl rollout status deployment/api --timeout=60s
```

Observez un nouveau Pod en `CreateContainerConfigError` à côté des deux anciens `Running`, et dans ses Events `Error: couldn't find key profil in ConfigMap lab14/api-config` ; puis le retour en arrière.

Explication. Les références sont résolues à la création du conteneur : une clé absente bloque le Pod, et le rollout du labo 12 garde les anciens Pods en service.

Nettoyage

```
kubectl delete namespace lab14
kubectl config set-context --current --namespace=default
kubectl get configmaps -A | grep -c lab14
```

Observez `0`. ConfigMaps, Secret, Deployments et Services sont partis avec le namespace. Les fichiers de `~/labo-docker/14/` servent de base au labo 15, qui ajoute la base de données.

Ce que vous devez pouvoir affirmer maintenant

- `apply -f`, `--from-file`, `--from-literal`, `--dry-run=client -o yaml` : quatre façons de créer ConfigMaps et Secrets.
- `stringData` devient `UzNjcjN0LXBn` ; `base64 -d` rend `S3cr3t-pg` : pas un chiffrement.
- `envFrom` → `APP_MESSAGE` dans `env` ; volume `secret` → `/run/secrets/db/POSTGRES_PASSWORD`, absent de l'environnement.
- `nginx:alpine` officiel configuré par deux ConfigMaps montées : `default.conf -> ..data/default.conf`.
- ConfigMap modifiée : variable inchangée jusqu'au `rollout restart`, fichier mis à jour seul en une à deux minutes.
- Clé manquante : `CreateContainerConfigError`, `couldn't find key profil`, anciens Pods intacts.